

# OPERATING SYSTEMS AND SYSTEMS PROGRAMMING

CODE : 25CS2104E

ROLL NO : 2520030255

NAME : V SAI NIKHIL

## WEEK-1 PRACTICAL

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using `fork()`. 3. Execute the command in the child process using an appropriate `exec()` system call. 4. Allow the parent process to wait for the child using `wait ()`. 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (`uname`, `lscpu`, `lsblk`, `ps`, `top`), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char command[100];

    printf("Enter Linux command: ");
    scanf("%s", command);

    pid_t pid = fork();

    if (pid == 0) {
        // Child process
        printf("Child PID: %d\n", getpid());
        execlp(command, command, NULL);

        printf("Command execution failed\n");
    }
    else {
        // Parent process
        printf("Parent PID: %d\n", getpid());
        wait(NULL);
        printf("Child process completed\n");
    }

    return 0;
}
```

OUTPUT :

```
vsainikhil@NIKHIL-BOOK:~$ vi program.c
vsainikhil@NIKHIL-BOOK:~$ gcc program.c -o program
vsainikhil@NIKHIL-BOOK:~$ ./program
Enter Linux command: ls
Parent PID: 564
Child PID: 565
a.out      addtion.c  develop   develop1.c  example.c  execworking.c  exeworking.c  operations.c  p2      process.c  switchcase
addition   code1      develop.c  dir2        execl      execworking.c.save  file1.txt     p1          p2.c     program   switchcase.c
addition.c code1.c    develop1   example     execl.c    exeworking      operations    p1.c        process  program.c
Child process completed
vsainikhil@NIKHIL-BOOK:~$ ./program
```